Machine Learning II

Data Science M.Sc.

Prof. Dr. Steffen Wagner
Berliner Hochschule für Technik

Winter term 2025/26

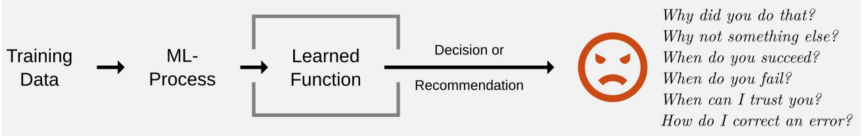
An Introduction to **Interpretable Machine Learning**

Deconstructing the *Black Box* with Explainable AI[1]

[1] We will use both the terms *Explainable AI (XAI)* and *Interpretable ML (IML)* synonymously.

Why Interpretability matters

Often a ML process produces a learned function that is a black box. This leads to critical *unanswered* questions from the user.



Critical scenarios: a ML model …

- denies a loan.
- flags a tumor.
- recommends a prison sentence[2].
- … .

[2] Ryberg, J. (2025). *Artificial intelligence at sentencing: when do algorithms perform well enough to replace humans?*. AI and Ethics, 5(2), 1009-1018. https://link.springer.com/content/pdf/10.1007/s43681-024-00442-5.pdf

Why Interpretability matters

Identification of incorrect mechanisms

Why Interpretability matters

Identification of incorrect mechanisms

Why Interpretability matters

Identification of incorrect mechanisms

Classified as horse

Not classified as horse

Why Interpretability matters

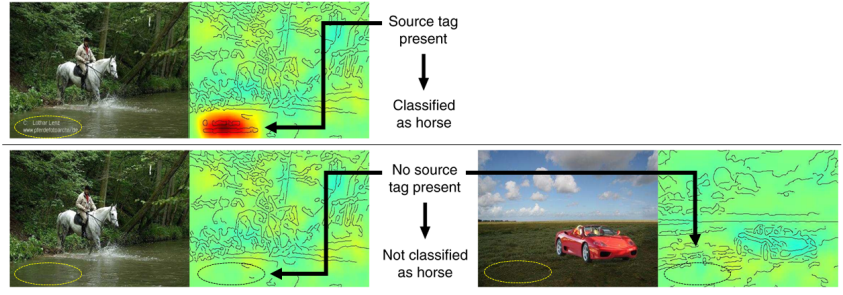
Identification of incorrect mechanisms

Classified as horse

Not classified as horse

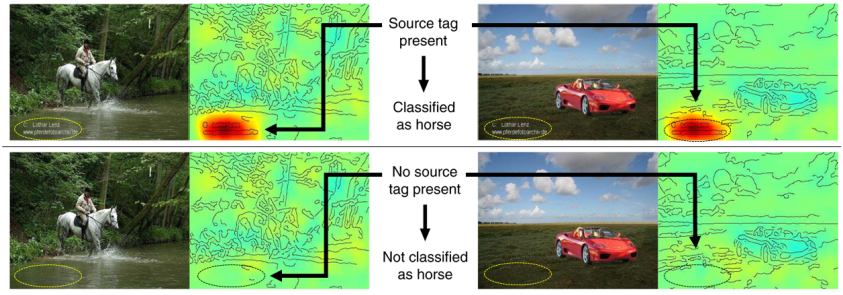
Why Interpretability matters

Identification of incorrect mechanisms



Why Interpretability matters

Identification of incorrect mechanisms



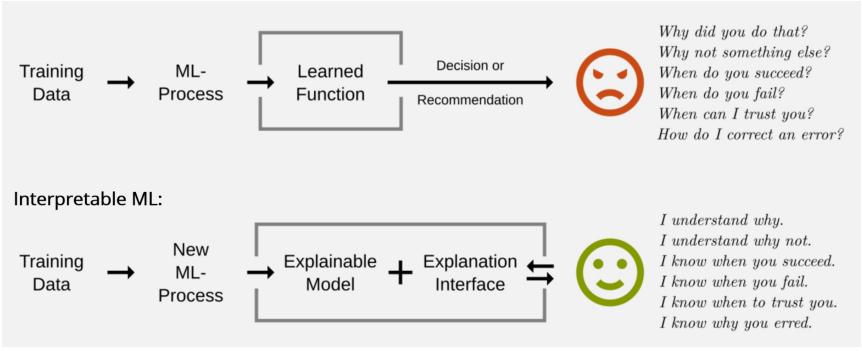
Sebastian Lapuschkin et al. *Unmasking clever hans predictors and assessing what machines really learn.* Nature communications, 10(1), 2019.
https://www.nature.com/articles/s41467-019-08987-4

The Goals of Interpretability

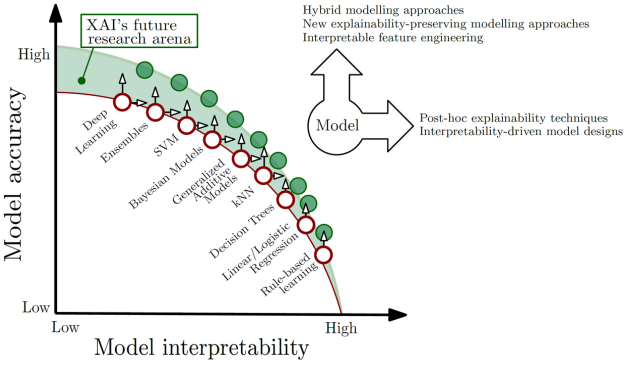
1. **Improving the Model**
   - **Debugging:** Find "Clever Hans" predictors, i.e. models that "cheat" using shortcuts.
   - **Feature Engineering:** Use feature insights to get new ideas for creating better features.
   - **Sanity Checks:** Verify that feature effects match domain knowledge (e.g., "Does 'Age' impact risk in a way that makes sense?").
2. **Justifying the Model and its Predictions**
   - **Build Trust:** Justify predictions to various stakeholders (managers, clients, regulators).
   - **Enable Recourse:** Allow a "decision subject" (e.g., a loan applicant) to understand *why* a decision was made and how they might contest it.
   - **Regulatory Compliance:** Meet legal and regulatory requirements (like GDPR's "Right to Explanation") by proving a model is safe and fair.
3. **Discovering New Insights**
   - **Knowledge Generation:** Use the model to learn new things about the world.
   - **Inform Decisions:** Move beyond just *predicting* churn to *understanding* what drives it, e.g. allowing a marketing team to take targeted action.

Source: Based on "Goals of Interpretability" from Christoph Molnar: `https://christophm.github.io/interpretable-ml-book/goals.html`

From Predictive ML to Interpretable ML

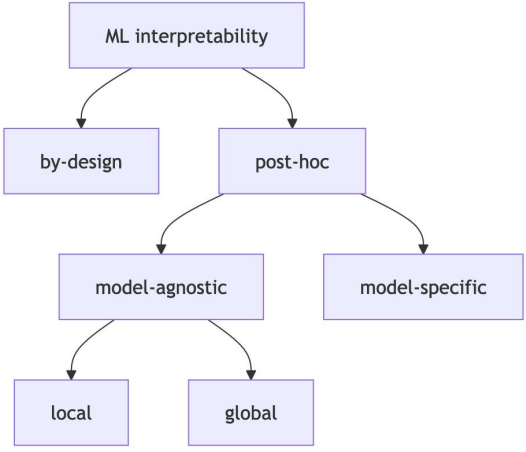


The Performance-Interpretability Trade-off



Source: Arrieta et al: *Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI.* Information Fusion. 2020. (`https://doi.org/10.1016/j.inffus.2019.12.012`)

A Taxonomy of Interpretability Methods



Source: "Methods overview" from Christoph Molnar: `https://christophm.github.io/interpretable-ml-book/overview.html`

Interpretable models

Interpretability by Design (Intrinsic): White box models

- We choose a ML algorithm that is based on a representation *understandable* by humans.
- The model itself is the explanation, i.e. model and/or its parameters can be interpreted.

Examples:

- Linear / Logistic Regression: A one-unit increase in x_j changes the outcome y by β_j, holding all else constant.
- GAMs: The visualized splines show the relationships between x and y
- Decision Trees: A set of explicit `IF-THEN-ELSE` rules partitioning the feature space.
- Decision Rules: The rules themselves.

Post-hoc Interpretability

We first train a complex, “black box” model (like Random Forests, XGBoost or Neural Network) and then apply a separate method *afterwards* to explain it. We distinguish:

Model-Specific post-hoc methods

- Can only be used for a specific model class.
- Example: Gini importance in a Random Forest.

Model-Agnostic post-hoc methods

- Can be used on *any* machine learning model, regardless of its internal structure.
- This is the most flexible and powerful category.
- Examples: PDP, SHAP, Permutation Importance.

Scopes of Interpretability

Global Methods:

- Explain the *entire model's* behaviour.
- Questions:
  - **Feature importance:**
    Which features are carrying the most important informations?
    Which features contribute the most to model quality?
  - **Feature effects:**
    What is the relationship between feature X and outcome Y?
    How does X affect Y?

Local Methods:

- Explain a *single, individual prediction*.
- Question: Why was *this specific customer* denied a loan?

Model-Agnostic XAI

In the following, we focus on **post-hoc model-agnostic**[3] methods.

Advantages:

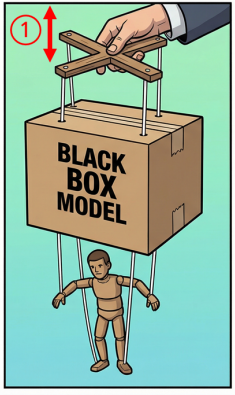
- **Flexibility:** Use any ML model you like (e.g., XGBoost, Neural Net).
- **Standardization:** The interpretation method is the same.
- **Comparability:** Different models can be evaluated by the same XAI method.

General Idea:

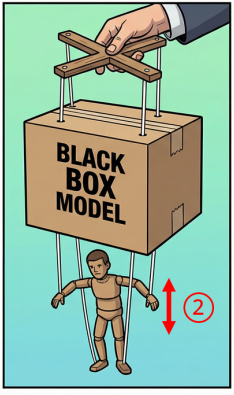
- We only need the **Prediction Function** $\hat{y} = f(\mathbf{X})$ of our trained (black box) model. We don't need to understand and know internal model parameters.
- We gain understanding by investigating how a change in input feature values changes the model's outcome, i.e. its predictions.

[3] In the very end of this chapter we will come back to one model-specific method.

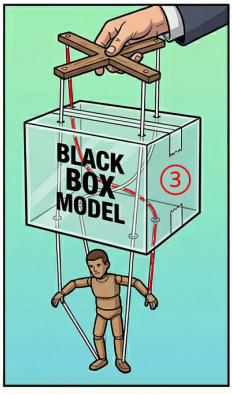
Model-Agnostic XAI



Vary input features



Observe how predictions change



Understand model mechanism

Overview of Model-Agnostic XAI Methods

Method	Global Feature Importance	Global Feature Effects	Local Explanation
Permutation Feature Importance (PFI)	**Yes** (Provides a ranked list of what's important)	**No** (Tells you *if* a feature matters, not *how*)	**No**
Partial Dependence Plot (PDP)	**Indirectly** (not recommended)	**Yes** (Shows the average "what-if" effect. **Note: Unreliable with correlated features**)	**No**
Accumulated Local Effects (ALE)	**Indirectly** (not recommended)	**Yes** (Shows the average effect. **Designed to be robust to correlated features**)	**No**
SHAP	**Yes** (By aggregating local values)	**Yes** (Using summary and dependence plots)	**Yes** (This is its foundation)

Remarks:

- The table shows why SHAP is so popular. It is a unified framework that provides a clear solution for all three goals: it builds local explanations which can then be aggregated to create robust global feature importance and global feature effects.
- ALE is the modern, preferred alternative to PDP for only analyzing global feature effects, as it correctly handles correlated features.

Permutation Feature Importance (PFI)

Core Idea: How much does the model's performance get worse when we *break* a feature's link to the target by randomly shuffling it?

Algorithm:

1. Train ML model and calculate its baseline test error e_{orig} using an appropriate loss function L.
2. For *selected* feature S create a *broken* dataset $\mathbf{X}_{S,\text{perm}} = (S, \mathbf{C})$ by **randomly shuffling** values in feature S and leaving all other *complementary* features $\mathbf{C}$ unchanged.
3. Calculate the new error on this permuted data: $e_{S,\text{perm}} = L(y, f(\mathbf{X}_{S,\text{perm}}))$.
4. The importance of feature S is the **difference** (or ratio) between the errors:

$$FI_S = \Delta e_S = e_{S,\text{perm}} - e_{\text{orig}}$$

5. Do so for every feature and compare the FI values.

Remarks:

- **Pros:** Model-agnostic, very intuitive, provides a single global importance score.
- **Cons: Problematic with correlated features**.

Partial Dependence Plot (PDP)

Core Idea: What is the *average* model prediction as one feature changes?

Algorithm:

1. Choose a feature S and a grid of values for it (usually covering its range).
2. To get the PDP value for a specific grid value of $S = s^*$:
   - Take *all n observations* from your dataset.
   - **Force** the column S to the specific grid value s^* for *every single observation*, leaving all other features $\mathbf{C}$ as-is.
   - Calculate the **average** of all n predictions.
3. Plot these averages for each grid value.
4. The PDP value for a feature value s^* is:

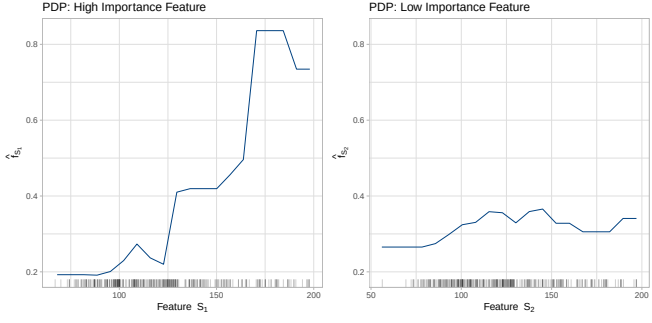
$$\hat{f}_S(s^*) = \frac{1}{n} \sum_{i=1}^{n} f(S = s^*, \mathbf{C}_i)$$

where $\mathbf{C}_i$ are the values of all *other* features $\mathbf{C}$ for the i-th observation.

Deriving Feature Importance from PDP

Core Idea: A feature is "important" if changing its value causes a *large change* in the model's average prediction.

- A feature with a **flat PDP line** is **unimportant** because changing its value doesn't affect the model's output.
- A feature with a **steep or "wiggly" PDP line** is **important**.
- We quantify "wiggliness" by measuring the **dispersion** of the PDP values.



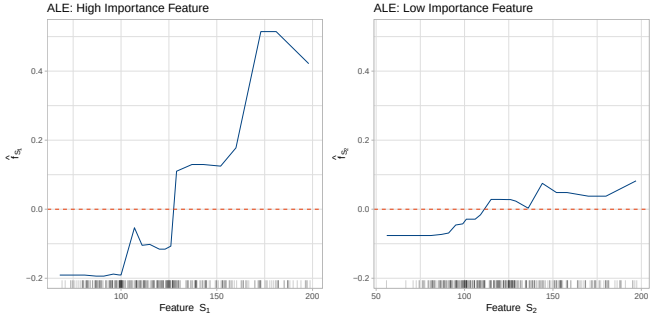
Accumulated Local Effects (ALE)

Core Idea: A robust alternative to PDP that **handles correlated features** by averaging *changes* in predictions, not the predictions themselves.

Algorithm:

1. Select a feature S and divide its range into k small intervals $I_k = (s_k^{low}, s_k^{upp}]$.
2. Use *only* the data points i with $s_i \in I_k$:
   i) By replacing s_i with the interval's boundaries, Calculate the i **differences in predictions**
   $$\Delta f(s_i) = f(s_k^{upp}, \mathbf{C}_i) - f(s_k^{low}, \mathbf{C}_i)$$
   ii) For an interval k, $\Delta f(s_k)$ is the average of all $\Delta f(s_i)$:
   $$\Delta f(s_k) = \frac{1}{N_i} \sum_i \Delta f(s_i)$$
   iii) Do so for all intervals.
3. **Accumulate** these average differences $\Delta f(s_k)$ across all intervals to build the ALE plot:
   $$f(s) = \sum_k \Delta f(s_k)$$
4. Center $f(s)$, so that the mean effect is zero.

Accumulated Local Effects (cont)



Remarks:

- **Pros: Robust to correlated features**, fast
- **Cons:** Slightly more complex to explain than PDP, ALE can be interpreted as the main effect of the feature at a certain value compared to the average prediction of the data.
- Feature Importance: Can be inferred from the plot's effect size as with PDP

- Lundberg et al: *A unified approach to interpreting model predictions.* Advances in neural information processing systems. 2017.
  `https://proceedings.neurips.cc/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf`
- `https://github.com/slundberg/shap`

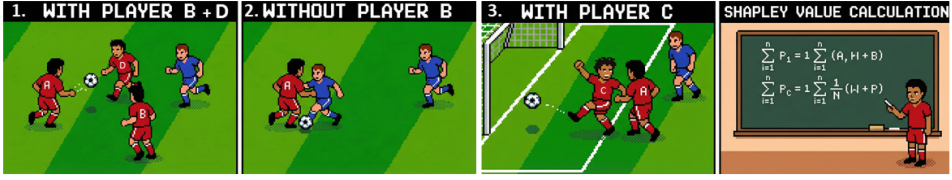
Why SHAP?

SHAP (SHapley Additive exPlanations) has become an industry standard.

- It is a framework to explain **individual predictions** (local).
- Its global interpretation methods are built by aggregating these individual effects.
- This provides a **common ground for local and global explanations**.
- It has a **solid theoretical foundation** based on Shapley values from coalitional game theory.
- SHAP is a framework to estimate Shapley values in the XAI context and marks a change in popularity and usage of Shapley values for XAI.
- The SHAP concept is **model-agnostic**.

Shapley values and coalitional game theory

Consider a soccer match and a coalition of players A, B, C and D:



1. Players A, B and D are working together to advance the ball down the soccer field.
2. Player A is without team-mates continuing the attack on its own.
3. Player A is joined by Player C. A passes to C and C scores.

Question: What is the contribution of player A achieving the goal?

Coalitional game theory

- **The “Game”:** A cooperative effort where multiple “players”, i.e. the coalition, work together to achieve a single outcome or “payout”.
- **The “Payout”:** The total value generated by the coalition.
- **The “Players”:** The specific participants in a specific game.
- **The Goal:** To distribute the total payout fairly among the players based on their individual contributions.

- **The Solution (Shapley Value):**
  It calculates the **average marginal contribution** of a player across all possible coalitions.

Mapping Game Theory to Machine Learning

Game Theory Concept	Machine Learning Equivalent
The Game	The model making a prediction for a single instance.
The Players	The **feature values** of that instance (e.g., `Age`=30, `Income`=50k).
The Payout	The difference between the **actual prediction** and the **average prediction** ($f(x) - E[f(x)]$).
Coalition	A subset of features (some feature values are "present", others are "absent" or marginalized).
Marginal Contribution	The change in prediction when a specific feature is added to a coalition.
Shapley Value	The feature's contribution to the prediction (its "importance" for this specific instance).

Shapley values: a ML example

Example: **apartment price** predictions[4]

Features:

- `park_near_by` (yes/no)
- `flat_size` (square meters)
- `floor` (1st, 2nd, 3rd, ...)
- `cats_allowed` (yes/no)

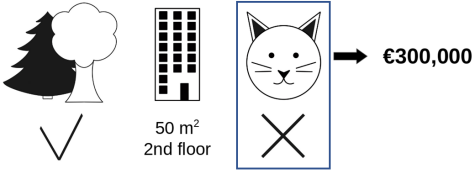
We consider a specific flat (with park near by, 50 m^2, 2nd floor and no cats allowed).

Questions:
To what extent does the fact that cats are not allowed contribute to the predicted price of 300.000 EUR?

[4] source: `https://christophm.github.io/interpretable-ml-book/shapley.html`

Shapley values: a ML example

Example: **apartment price** predictions[5]



What is the contribution of `no-cats-allowed` to the predicted price?

[5] source: `https://christophm.github.io/interpretable-ml-book/shapley.html`

Shapley values: a ML example

Definition: The Shapley value is the average marginal contribution of a feature value across **all possible coalitions** of features.

# Features	# Coalitions	Sets of coalitions **S** without feature **X**:
0	1	
1	3	50m² | 2nd floor
2	3	50m² | 2nd floor | 50m² 2nd floor
3	1	50m² 2nd floor

Shapley values: a ML example

Definition: The Shapley value is the **average marginal contribution** of a feature value across all possible coalitions of features.

# Features	# Coalitions	Sets of coalitions **S** without feature **X**:	Marginal contribution of **X**	Weighted Average
0	1		 $= \Delta_{\lvert S\rvert=0}$	$\Delta_{\lvert S\rvert=0}$
1	3		 $= \Delta_{\lvert S\rvert=1}$	$\Delta_{\lvert S\rvert=1}$
2	3		 $= \Delta_{\lvert S\rvert=2}$	$\Delta_{\lvert S\rvert=2}$
3	1		 $= \Delta_{\lvert S\rvert=3}$	$\Delta_{\lvert S\rvert=3}$

Φ

Shapley values for ML models

The vector of Shapley values $\Phi(X_j)$ of a feature X_j are its contribution to the predicted payout, weighted and summed over all possible feature value combinations:

$$\Phi(X_j) = \sum_{S \subseteq N \setminus X_j} \underbrace{\frac{|S|!\,(|P| - |S| - 1)!}{|P|!}}_{\text{weighting}} \underbrace{[f(S \cup X_j) - f(S)]}_{=\Delta_S(X_j)}$$

with

$f(S \cup X_j)$ ... outcome of ML model for set of features S with consideration of feature X_j

$f(S)$ ... outcome of ML model for set of features S without consideration of feature X_j

P ... set of all features

$|P|$ ... number of all features

S ... subset of features in specific coalition

$|S|$ ... number of features in coalition S

X_j ... feature of interest

Estimation of Shapley values

- All possible coalitions S of feature values have to be evaluated with and without the j-th feature to calculate the exact Shapley value.
- For more than a few features, the exact solution to this problem becomes problematic as the number of possible coalitions exponentially increases as more features are added.
- It requires retraining the model on $2^{|P|}$ feature coalitions.
- **SHAP** provides efficient way(s) to estimate Shapley values.

Estimation of Shapley values

There are three ways to estimate Shapley values based on SHAP for tabular data:

- KernelSHAP: The original estimation method based on a theoretical connection to LIME[6]. Since it is slow, it isn't used any more in most software packages.
- Permutation Method:[7] Most efficient **model-agnostic** estimation method. The idea is to sample cleverly from all possible coalitions by only considering a small number of sets with a permuted feature sequence. Default model-agnostic software implementation (e.g. `DALEX` package in R).
- TreeSHAP[8]: TreeSHAP was introduced as a fast, **model-specific** alternative to KernelSHAP. Integrated directly into boosting frameworks like XGBoost, Catboost and LightGBM.

We will briefly discuss TreeSHAP, because
Gradient Boosted Trees + TreeSHAP = "Industry Standard".

[6] = Local Interpretable Model-agnostic Explanations. Not covered in this lecture.

[7] Mitchell, Rory, Joshua Cooper, Eibe Frank, and Geoffrey Holmes. 2022. "Sampling Permutations for Shapley Value Estimation." Journal of Machine Learning Research 23 (43): 1–46. `http://jmlr.org/papers/v23/21-0439.html`.

[8] Lundberg, Scott M., Gabriel G. Erion, and Su-In Lee. 2019. "Consistent Individualized Feature Attribution for Tree Ensembles." arXiv. `https://doi.org/10.48550/arXiv.1802.03888`.

TreeSHAP

- This is a **fast, model-specific** alternative for tree-based models (XGBoost, Random Forest) leveraging the tree structure for efficient computation.
- It computes **exact Shapley values** and reduces the computational complexity from $O(TL2^{|P|})$ to $O(TLD^2)$, where T is the number of trees, L is the maximum number of leaves in any tree, and D is the maximal depth of any tree.[9]
- Since the number of considered features in a single tree is limited by the number of knots, a majority of features is absent in a specific tree. Therefore only a small set of coalitions S alter the predictions, i.e. influence the Shapley values.
- TreeSHAP only focuses on this small set of relevant coalitions.

[9] Especially for Gradient Boosted Trees we use weak learners, i.e. tree stumps, with very small D.

SHAP results

Outcome: Averaged contribution Φ_{ij} to $\hat{y}_i = f(\mathbf{X}_i)$ for every observation i and feature j:

	feature 1	feature 2		feature p
obs 1	$\Phi_{1,1}$	$\Phi_{1,2}$	$\dots$	$\Phi_{1,p}$
obs 2	$\Phi_{2,1}$	$\Phi_{2,2}$	$\dots$	$\Phi_{2,p}$
	$\vdots$	$\vdots$	$\ddots$	
obs n	$\Phi_{n,1}$	$\Phi_{n,2}$		$\Phi_{n,p}$

Learnings:

- row values = **microscopic explanations**: how does each feature $x_{i,\cdot}$ contribute to a specific prediction $\hat{y}_i$
- column values = **feature effects**: how do the contributions $\Phi_{\cdot,j}$ depend on feature values $x_{\cdot,j}$
- sum of absolute column values = **feature importance**: inferred from feature effect size

SHAP: example

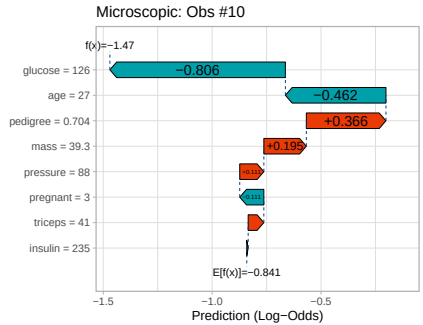
- XGBoost model for the prediction of a binary outcome (`diabetes: yes/no`).
  See `?mlbench::PimaIndiansDiabetes2` for data documentation and exercise sheet covering XGBoost.
- Data set with 768 observations and 8 features.
- Shapley values can be explicitly obtained via

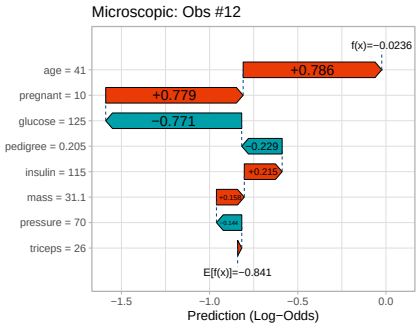
```
xgboost:::predict.xgb.Booster(..., predcontrib = TRUE)
```

	pregnant	glucose	pressure	triceps	insulin	mass	pedigree	age	BIAS
[1,]	-0.046	-1.068	-0.080	-0.196	0.073	-0.190	-0.462	-1.032	-0.841
[2,]	0.146	0.847	0.260	0.109	0.307	0.278	0.643	0.360	-0.841
[3,]	-0.007	-0.942	0.502	0.153	0.518	0.203	-0.065	-0.084	-0.841
[4,]	-0.079	1.910	-0.088	0.068	0.267	0.090	-0.043	0.578	-0.841
[5,]	-0.058	2.598	-0.014	-0.013	0.047	0.077	-0.107	0.411	-0.841
[6,]	-0.178	2.965	-0.086	0.000	0.327	-0.679	0.076	0.356	-0.841
[7,]	0.075	-0.283	0.136	0.037	0.209	0.892	0.135	0.453	-0.841
[8,]	-0.141	-1.074	0.263	-0.017	-0.889	0.136	-0.491	0.142	-0.841
[9,]	0.041	-0.469	-0.074	0.017	0.395	0.287	0.465	0.546	-0.841
[10,]	-0.111	-0.806	0.111	0.072	0.006	0.195	0.366	-0.462	-0.841
[11,]	0.581	0.146	0.169	0.079	0.441	0.339	-0.198	0.877	-0.841
[12,]	0.779	-0.771	-0.144	0.023	0.215	0.158	-0.229	0.786	-0.841

- Last column `BIAS` contains the overall mean $E[f(x)] = \bar{y}$. The other columns showing the Shapley values are related to this mean.

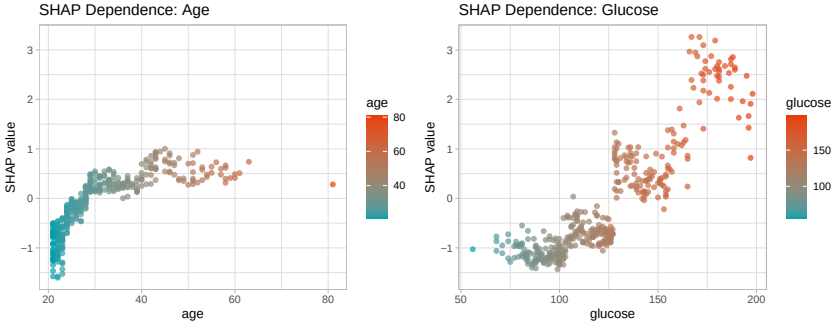
Microscopic explanations: waterfall plots





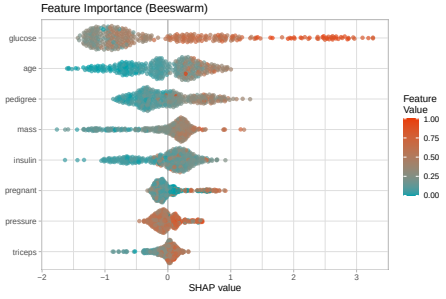
- The prediction $\hat{f}(x_{10}) = -1.47$ and $\hat{f}(x_{12}) = -0.024$.
- The difference between $\hat{f}(x_i)$ and $E[f(x)]$ is the sum of the individual Shapley values.
- The observed feature values x_{ij} are given as numeric values on the plot's y-axis.
- Plots are obtained using the `shapviz` package with which you will work in the exercise session.

Feature Dependence



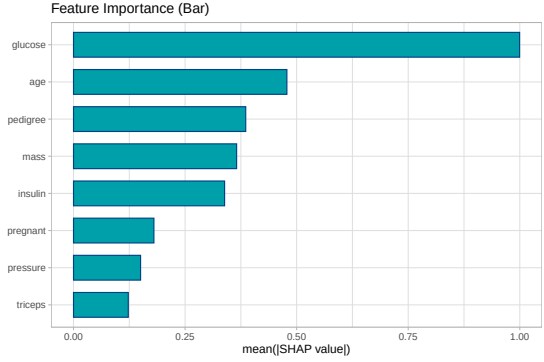
- SHAP values on the y-axis plotted against observed feature values on the x-axis.
- The dependence between feature X_j and the predicted outcome is visualized.
- The larger the effect size, i.e. the spread of SHAP values on the y-axis, the more the feature contributes to the variation in $\hat{y}_i$.

Feature Importance: Beeswarm Plot



- The features are listed from top to bottom based on their overall importance.
- The color represents the actual feature value (Blue = Low, Red = High). By looking at where the Red dots land, you can see the direction of the relationship: if Red dots are on the right, higher feature values increase the predicted outcome.
- The horizontal spread of the dots shows the variance of the effect. A wide spread indicates that the feature acts differently for different patients, while a tight cluster suggests an almost consistent effect for all observations.

Feature Importance: Bar Plot



- Feature importance is obtained as mean of absolute Shapley values for each feature.
- The larger the effect size, the larger the feature importance.

SHAP

Consistent concept for *local* and *global* explanations:

Summary

- **Interpretability is key** for model acceptance and improvement.
- There is a **trade-off** between predictive power and interpretability.
- **Post-hoc techniques** (like PFI, PDP, ALE, LIME, and SHAP) shed light on black box algorithms.
- The **SHAP framework** is a powerful, unified approach providing Local & global explanations.
- The combination of **Gradient Boosted Trees** and **TreeSHAP** is SOTA.

IML vs. Statistical Inference

- IML methods are **descriptive tools**. They describe the inner workings of the trained machine learning model, not the statistical properties of the population from which the data was drawn.
- Unlike classical statistical models, post-hoc IML results do not provide **p-values**, **standard errors**, or **hypothesis tests**.
- One can only assume that a feature will be very likely *"statistically significant"* simply if it has a high importance score or shows a non-flat PDP curve.